



icev Instituto de
Ensino Superior



Aula

Automação de Testes: Introdução e Práticas com Selenium

O que é Automação de Testes?

Definição

Processo de usar ferramentas e scripts para executar testes de software automaticamente.

Vantagens

- Maior cobertura de testes
- Execução mais rápida e eficiente
- Repetibilidade sem erro humano

Desafios

- Manutenção de scripts
- Necessidade de conhecimento técnico





Por Que Automatizar Testes?

Por Que Automatizar Testes?

Para não testar manualmente mais

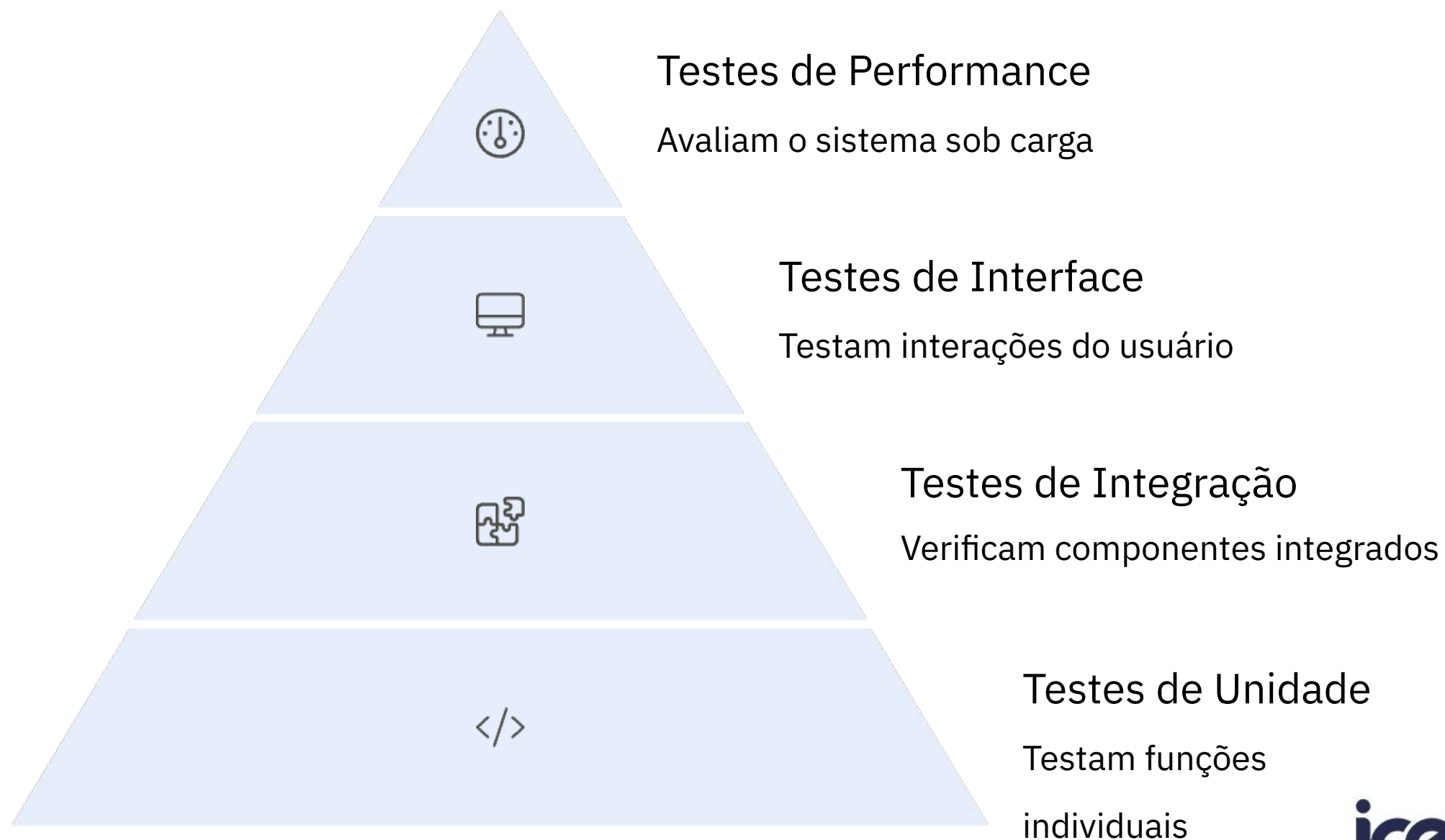


Por Que Automatizar Testes?

Vou
demitir
esse cara

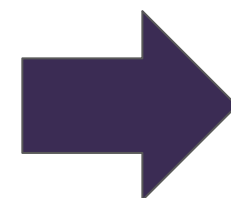


Tipos de Testes Automáticos



Quem conhece e já assistiu...?

INDIANA JONES™



Relíquia do Destino



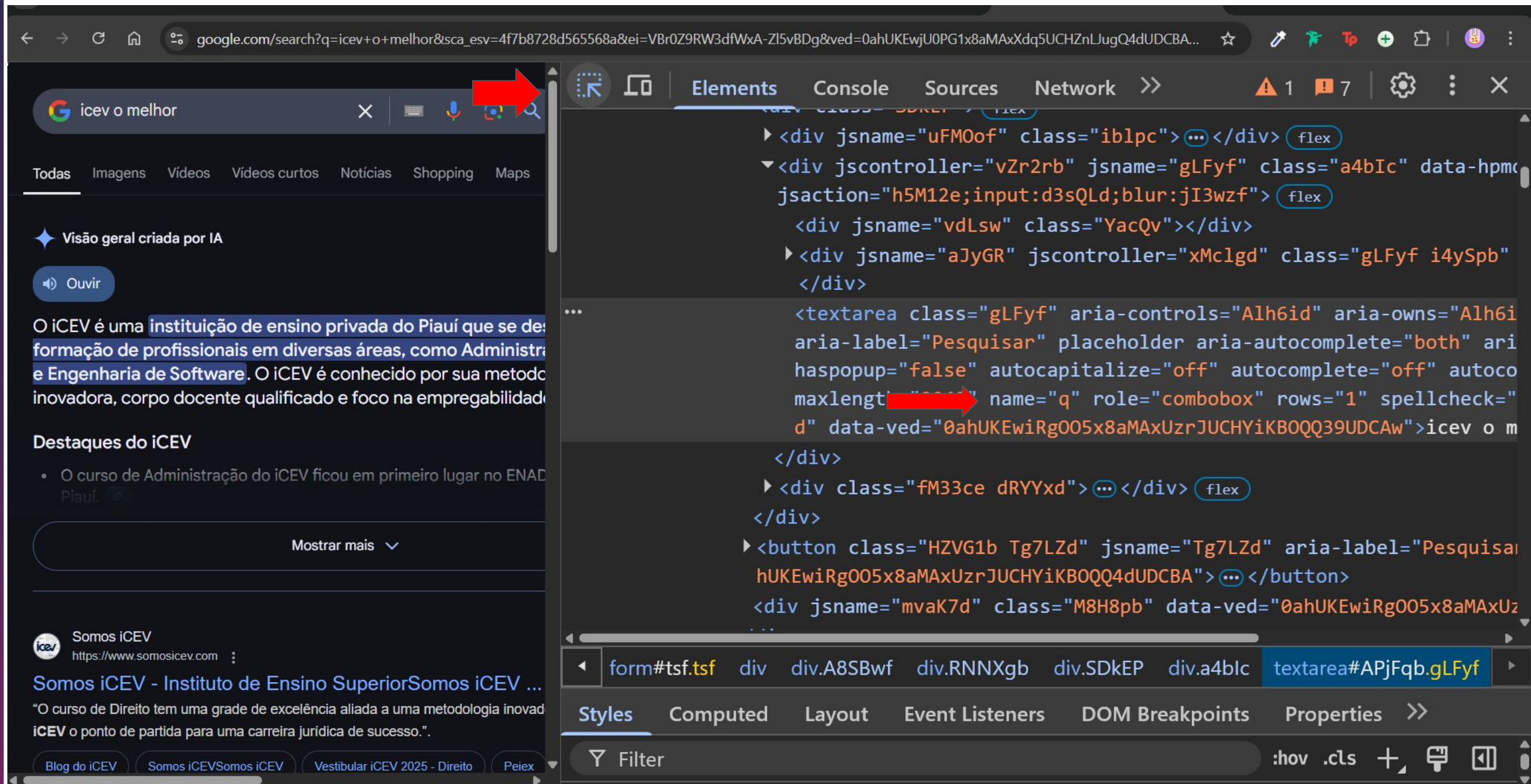
Relíquia do Destino

Mapa do Tesouro



Localização = Id, Xpath, Classe ...

Como eu encontro esses elementos?



The image shows a Google search result for "icev o melhor" in a Chrome browser. The search result includes a snippet about iCEV (Instituto de Ensino Superior do Piauí) and a search bar. The Chrome DevTools "Elements" panel is open, displaying the HTML structure of the search results. A red arrow points from the search bar in the browser to the corresponding HTML element in the DevTools panel.

Search Results Snippet:

Visão geral criada por IA

Ouvir

O iCEV é uma instituição de ensino privada do Piauí que se dedica à formação de profissionais em diversas áreas, como Administração e Engenharia de Software. O iCEV é conhecido por sua metodologia inovadora, corpo docente qualificado e foco na empregabilidade.

Destaques do iCEV

- O curso de Administração do iCEV ficou em primeiro lugar no ENADE do Piauí.

Mostrar mais

Somos iCEV
https://www.somosicev.com

Somos iCEV - Instituto de Ensino Superior

"O curso de Direito tem uma grade de excelência aliada a uma metodologia inovadora. O iCEV é o ponto de partida para uma carreira jurídica de sucesso."

Blog do iCEV | Somos iCEV | Vestibular iCEV 2025 - Direito | Peixe

HTML Structure (Elements Panel):

```
<div jsname="uFM0of" class="ib1pc">...</div>
<div jscontroller="vZr2rb" jsname="gLfyf" class="a4bIc" data-hpme...
  jsaction="h5M12e;input:d3sQLd;blur:jI3wzf">
    <div jsname="vdLsw" class="YacQv"></div>
    <div jsname="aJyGR" jscontroller="xMclgd" class="gLfyf i4ySpb">
      <div jsname="mvaK7d" class="M8H8pb" data-ved="0ahUKEwiRg005x8aMAxUzrJUCHYiKBOQQ39UDCAw">icev o m
    </div>
  </div>
</div>
<div class="fM33ce dRYYxd">
  <div class="HZVG1b Tg7LZd" jsname="Tg7LZd" aria-label="Pesquisar" data-ved="0ahUKEwiRg005x8aMAxUzrJUCHYiKBOQQ4dUDCBA">
    <button class="HZVG1b Tg7LZd" jsname="Tg7LZd" aria-label="Pesquisar" data-ved="0ahUKEwiRg005x8aMAxUzrJUCHYiKBOQQ4dUDCBA">
      <div jsname="mvaK7d" class="M8H8pb" data-ved="0ahUKEwiRg005x8aMAxUzrJUCHYiKBOQQ4dUDCBA">icev o m
    </div>
  </div>
</div>
```

The breadcrumb trail at the bottom of the Elements panel shows the path: `form#tsf.tsf > div > div.A8SBwf > div.RNNXgb > div.SDkEP > div.a4blc > textarea#APjFqb.gLfyf`.

Preparando o Ambiente

Verificar Python

Abra o terminal e digite:

`python --version`

Se não estiver instalado, baixe em python.org/downloads

Instalar Selenium

Execute: `pip install selenium`

Verificar Pip

Digite: `pip --version`

Caso não tenha, baixe `get-pip.py` e execute-o

Instalar WebDriver Manager

Execute: `pip install
webdriver-manager`

SUCCESSFULLY INSTALLED

PYTHON



Drivers de Navegadores

WebDriver Manager

Gerencia automaticamente o download dos drivers necessários.

Recomendado para facilitar a configuração.

Download Manual

Chrome: sites.google.com/a/chromium.org/chromedriver

Firefox: github.com/mozilla/geckodriver/releases

Coloque o driver em uma pasta acessível e configure o PATH.

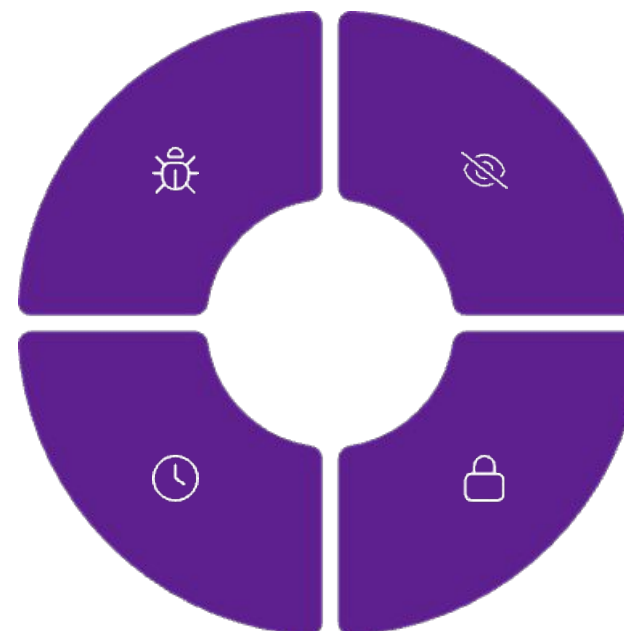
Solução de Problemas Comuns

Elementos Não Encontrados

Verifique seletores e aguarde carregamento da página.

Timeout

Aumente o tempo de espera com `implicitly_wait()`.



Elementos Invisíveis

Use wait conditions para aguardar visibilidade.

Elementos Desabilitados

Verifique o estado do elemento antes de interagir.

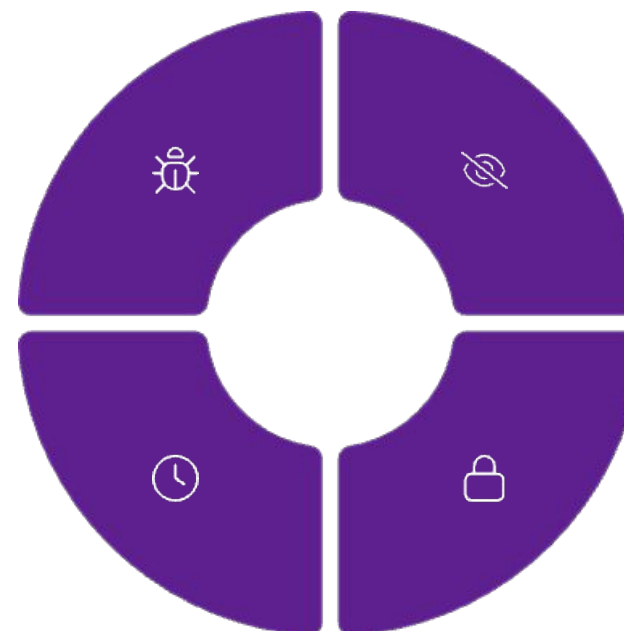
Solução de Problemas Comuns

Elementos Não Encontrados

Verifique seletores e aguarde carregamento da página.

Timeout

Aumente o tempo de espera com `implicitly_wait()`.



Elementos Invisíveis

Use wait conditions para aguardar visibilidade.

Elementos Desabilitados

Verifique o estado do elemento antes de interagir.

```
campo_pesquisa = WebDriverWait(driver, 10).until(
    EC.visibility_of_element_located((By.NAME, "q"))
)
```

Verificando a Instalação

Código de Teste

Crie um arquivo
teste_selenium.py com o
código para abrir o
navegador.

Importações

```
from selenium import webdriver  
from webdriver_manager.chrome import ChromeDriverManager
```

Execução

Inicialize o navegador, acesse uma página e feche após alguns segundos.

Interação com a Página



Preencher Campo

```
campo.send_keys("Texto de exemplo")
```



Clicar em Botão

```
botao = driver.find_element("id", "botao_id")
```

```
botao.click()
```



Fechar Navegador

```
driver.quit()
```

Clicando em um Campo



Inicializar WebDriver

```
driver = webdriver.Chrome()
```



Acessar Página

```
driver.get("https://www.exemplo.com")
```



Encontrar Campo

```
campo = driver.find_element("id", "campo_id")
```



Clicar no Campo

```
campo.click()
```

Comandos Básicos do Selenium

Inicializar WebDriver

```
from selenium import webdriver
```

```
driver = webdriver.Chrome()
```

Comandos de Navegação:

- `driver.get("url")` – Acessa uma página.
- `driver.back()` – Volta para a página anterior.
- `driver.forward()` – Avança para a próxima página.
- `driver.refresh()` – Recarrega a página.

Interação com Elementos:

- `element.click()` – Clica em um elemento.
- `element.send_keys("texto")` – Digita texto em um campo.
- `element.clear()` – Limpa um campo de texto.

Comandos Básicos do Selenium

Obter informações:

- `driver.title` – Retorna o título da página.
- `driver.current_url` – Retorna a URL atual.

Comandos de Espera:

- `driver.implicitly_wait(segundos)` – Espera implícita.
- `WebDriverWait(driver, tempo).until(...)` – Espera explícita.

Finalizar e Limpar:

- `driver.quit()` – Fecha o navegador completamente.
- `driver.close()` – Fecha apenas a aba atual.

Comandos Básicos do Selenium



Encontrar Elementos

`find_element(By.ID, "id_do_elemento")` – Pelo **ID**

`find_element(By.NAME, "nome_do_elemento")` – Pelo **name**

`find_element(By.CLASS_NAME, "classe")` – Pela **classe CSS**

`find_element(By.TAG_NAME, "tag")` – Pela **tag HTML**

Comandos Básicos do Selenium



Encontrar Elementos

`find_element(By.LINK_TEXT, "Texto do link")` – Pelo **texto do link**

`find_element(By.PARTIAL_LINK_TEXT, "parte do texto")` – **Por parte do texto do link**

`find_element(By.CSS_SELECTOR, "seletor_css")` – Usando **seletor CSS**

`find_element(By.XPATH, "caminho_xpath")` – Usando **XPath**

Importações no Selenium

```
from selenium.webdriver.common.by import By
```

- **O que é?:** Importa a classe `By` do Selenium, que permite localizar elementos na página da web.
- **Para que serve?:** Usada para identificar os elementos da página utilizando métodos como `By.ID`, `By.NAME`, `By.CLASS_NAME`, etc.
- **Exemplo:** `driver.find_element(By.ID, "username")` localiza o campo de entrada com o ID `username`.

Importações no Selenium

```
from selenium.webdriver.support.ui import WebDriverWait
```

- **O que é?:** Importa a classe `WebDriverWait` do Selenium, que ajuda a esperar até que um determinado evento aconteça na página.
- **Para que serve?:** Usada para aguardar até que um elemento esteja visível, clicável, ou tenha sido carregado antes de realizar a próxima ação.
- **Exemplo:** `WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.ID, "login-button")))` espera até que o botão de login esteja presente na página

Importações no Selenium

`from selenium.webdriver.support import expected_conditions as EC`

- **O que é?:** Importa a classe `expected_conditions` (comumente chamada de `EC`) do Selenium, que contém condições predefinidas para verificar o estado dos elementos.
- **Para que serve?:** Usada em conjunto com `WebDriverWait` para definir as condições de espera, como se o elemento está visível, clicável, ou presente na página.
- **Exemplo:** `EC.presence_of_element_located((By.ID, "login-button"))` verifica se o elemento de login está presente.

Importações no Selenium

```
from selenium.webdriver.chrome.service import Service
```

- **O que é?:** Importa a classe `Service` do Selenium, que é usada para gerenciar o serviço do WebDriver (neste caso, o ChromeDriver).
- **Para que serve?:** Usada para iniciar e gerenciar o serviço do driver de navegador (ChromeDriver).
- **Exemplo:** `service = Service(ChromeDriverManager().install())`
prepara o ChromeDriver para ser executado.

Importações no Selenium

```
from webdriver_manager.chrome import ChromeDriverManager
```

- **O que é?:** Importa a classe `ChromeDriverManager` do pacote `webdriver_manager`, que ajuda a gerenciar e fazer o download automático do `chromedriver` adequado para a versão do seu navegador.
- **Para que serve?:** Usada para garantir que o driver correto do Chrome seja instalado e utilizado, sem precisar baixá-lo manualmente.
- **Exemplo:** `ChromeDriverManager().install()` faz o download do `chromedriver` apropriado para o seu ambiente.

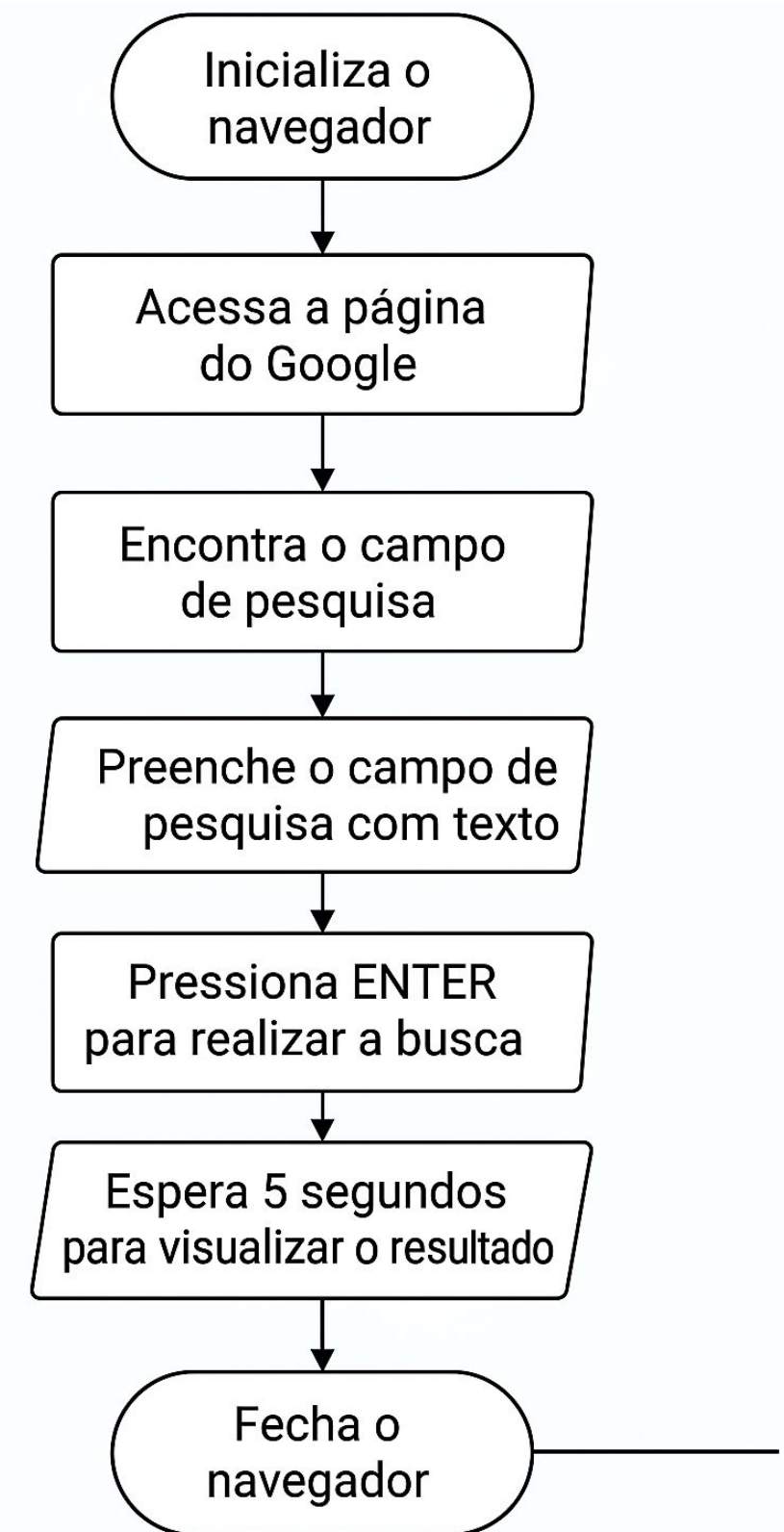
Importações no Selenium

```
from selenium import webdriver
```

- **O que é?:** Importa a biblioteca `webdriver` do Selenium, que fornece interfaces para interagir com o navegador e realizar testes automatizados.
- **Para que serve?:** Usada para criar uma instância do navegador (por exemplo, o Chrome) e controlar o fluxo do teste.
- **Exemplo:** `driver = webdriver.Chrome()` inicializa o navegador Chrome

Bora Praticar - Código Completo do Exercício

Façam *EM DUPLA* um código que acesse o google pesquise “Engenharia de Software no SomosICEV



Exercício Prático: Pesquisa no Google



Acessar Google

```
driver.get("https://www.google.com")
```



Encontrar Campo de Pesquisa

```
campo_pesquisa = driver.find_element("name", "q")
```



Preencher Campo

```
campo_pesquisa.send_keys("Engenharia de Software no SomosICEV")
```



Pressionar Enter

```
campo_pesquisa.send_keys(Keys.RETURN)
```

Preparação para a Automação de Testes

Ambiente de Desenvolvimento

Configure Python, Java ou outra linguagem de sua preferência.

Ferramenta de Automação

Instale Selenium, Cypress, Appium ou outra ferramenta adequada.

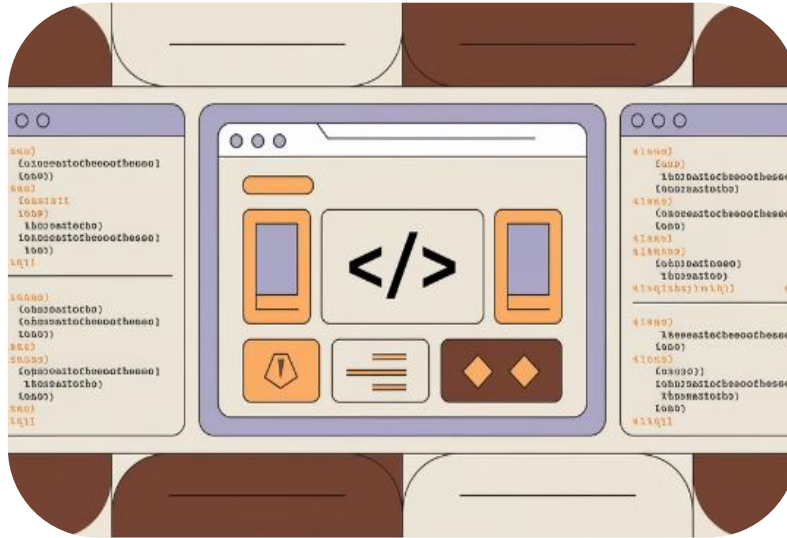
WebDriver

Necessário para controlar os navegadores de forma programática.

Framework de Testes

Configure TestNG, JUnit ou PyTest para estruturar seus testes.

O que é WebDriver?



Interface de Controle

O WebDriver permite interagir com navegadores de forma programática.



Simulação de Usuário

Simula ações reais como cliques, preenchimento de formulários e navegação.



Compatibilidade

Funciona com diversos navegadores como Chrome, Firefox e Edge.

Usando WebDriver Manager



Problema

WebDriver precisa ser atualizado e baixado manualmente com frequência.



Solução

WebDriver Manager automatiza o download e gerenciamento.



Benefício

Mantém seu ambiente sempre atualizado com a versão correta.

Como Instalar o WebDriver Manager

1º

Comando de Instalação

Use pip para instalar facilmente o pacote
webdriver-manager.

2º

Requisitos

Python e pip devem estar instalados em
seu sistema.

3º

Compatibilidade

Funciona com Chrome, Firefox, Edge e
outros navegadores.

```
pip install webdriver-manager
```

Automação do Login no Facebook (Visão Geral)

1º Abrir Navegador
Iniciar o Chrome com WebDriver

4º Submeter Formulário
Enviar dados de login



2º Acessar Facebook
Navegar até a página de login

3º Preencher Campos
Inserir email e senha

Exemplo de Código: Automação do Login

```
from selenium import webdriver
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys

# Configuração do WebDriver
driver = webdriver.Chrome(ChromeDriverManager().install())
driver.get("https://www.facebook.com/login")

driver.find_element(By.ID, 'email').send_keys("seu_email")
driver.find_element(By.ID, 'pass').send_keys("sua_senha")

# Enviando o formulário
driver.find_element(By.ID, 'pass').send_keys(Keys.RETURN)
```

Melhorando a Automação: Esperas Explícitas

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Espera até o campo de email aparecer
WebDriverWait(driver, 10).until(
    EC.presence_of_element_located((By.ID, 'email'))
)
```

Possíveis Erros e Soluções

Erro	Solução
Elemento não encontrado	Verifique o seletor do elemento (ID, XPATH)
Timeout	Use esperas explícitas para garantir carregamento
Seletor alterado	Use seletores mais robustos ou atualize scripts
Elemento não clicável	Espere até que o elemento esteja clicável

Não Expor Credenciais em Códigos

Segurança

Nunca deixe suas credenciais expostas diretamente no código.



Boas Práticas

Use variáveis de ambiente ou arquivos de configuração seguros.

Código Inseguro

Evite hardcoding de senhas como no exemplo do Facebook.



Riscos

Credenciais expostas podem ser comprometidas em repositórios públicos.



Como Proteger Suas Credenciais ?????

Como Proteger Suas Credenciais ????



Como Proteger Suas Credenciais

```
import os  
  
email = os.getenv("FB_EMAIL")  
  
password = os.getenv("FB_PASSWORD")
```

Use variáveis de ambiente para armazenar informações sensíveis. Isso mantém suas credenciais seguras e fora do código-fonte.

